

SOF 103 Week 12: Inheritance

Generated by Review Agent on 2026-07-02 06:36 UTC

Source: SOF 103-Chapter 12 - Inheritance2024_04b.ppt

Classification confidence: 80%

1. Core Knowledge Outline

中文大纲:

- **继承 (Inheritance) 的概念**
 - **为什么需要继承?** 避免代码重复, 建立类之间的层次关系 (is-a 关系)。
 - **核心思想:** 创建一个新类 (派生类/子类), 从一个或多个已存在的类 (基类/父类) 继承属性和行为。
- **C++ 继承的基本语法**
 - `class DerivedClass : access_specifier BaseClass { ... };`
 - **访问说明符 (Access Specifiers):** `public`, `protected`, `private`。它们决定了基类成员在派生类中的访问权限。
- **继承类型 (Types of Inheritance)**
 - **单继承 (Single Inheritance):** 一个派生类只有一个直接基类。
 - **多继承 (Multiple Inheritance):** 一个派生类有多个直接基类。
 - **多层继承 (Multilevel Inheritance):** 一个类从另一个派生类派生, 形成链式结构。
 - **层次继承 (Hierarchical Inheritance):** 多个派生类继承自同一个基类。
 - **混合继承 (Hybrid Inheritance):** 上述类型的组合, 常涉及菱形继承问题。
- **派生类中的构造函数和析构函数**
 - **构造函数:** 基类构造函数不会被继承。派生类构造函数需要显式调用基类构造函数来初始化基类部分。

- **析构函数**: 析构函数调用顺序与构造函数相反（先派生类，后基类）。基类析构函数通常应声明为 `virtual`。
- **函数重写 (Function Overriding) 与 虚函数 (Virtual Functions)**
 - **函数重写**: 在派生类中重新定义基类中已存在的成员函数。
 - **虚函数**: 使用 `virtual` 关键字声明的成员函数，允许通过基类指针或引用调用派生类版本的函数，实现**运行时多态 (Runtime Polymorphism)**。
- **纯虚函数与抽象类 (Pure Virtual Functions & Abstract Classes)**
 - **纯虚函数**: `virtual void func() = 0;`
 - **抽象类**: 包含至少一个纯虚函数的类。不能实例化，只能作为基类使用，为派生类提供接口规范。

English Outline:

- **Concept of Inheritance**
 - **Why Inheritance?** To avoid code duplication and establish hierarchical relationships (is-a relationship) between classes.
 - **Core Idea**: Creating a new class (derived class/child class) that inherits properties and behaviors from one or more existing classes (base class/parent class).
- **Basic Syntax of C++ Inheritance**
 - `class DerivedClass : access_specifier BaseClass { ... };`
 - **Access Specifiers**: `public`, `protected`, `private`. They determine the accessibility of base class members in the derived class.
- **Types of Inheritance**
 - **Single Inheritance**: One derived class has only one direct base class.
 - **Multiple Inheritance**: One derived class has multiple direct base classes.
 - **Multilevel Inheritance**: A class is derived from another derived class, forming a chain.
 - **Hierarchical Inheritance**: Multiple derived classes inherit from the same base class.
 - **Hybrid Inheritance**: A combination of the above types, often involving the diamond problem.

- **Constructors and Destructors in Derived Classes**

- **Constructors:** Base class constructors are not inherited. The derived class constructor must explicitly call a base class constructor to initialize the base class part.
- **Destructors:** Destructors are called in the reverse order of constructors (derived first, then base). Base class destructors should generally be declared `virtual`.

- **Function Overriding & Virtual Functions**

- **Function Overriding:** Redefining a member function of the base class in the derived class.
- **Virtual Functions:** Member functions declared with the `virtual` keyword, enabling calling the derived class version of a function through a base class pointer or reference, achieving **Runtime Polymorphism**.

- **Pure Virtual Functions & Abstract Classes**

- **Pure Virtual Function:** `virtual void func() = 0;`
- **Abstract Class:** A class containing at least one pure virtual function. Cannot be instantiated; serves only as a base class to define an interface for derived classes.

2. Key Concepts & Glossary

Term	Definition	Memory Anchor / Analogy
基类 / 父类 (Base Class / Parent Class)	被其他类继承的类。它定义了通用的属性和行为。	父母： 就像父母将基因（特征）遗传给孩子。基类是“父母”类。
派生类 / 子类 (Derived Class / Child Class)	从基类继承的类。它继承了基类的成员，并可以添加自己的新成员或修改继承来的行为。	孩子： 孩子继承了父母的基因，但也有自己独特的特征。派生类是“孩子”类。
访问说明符 (Access Specifier)	用于控制基类成员在派生类中访问权限的关键字（ <code>public</code> ， <code>protected</code> ， <code>private</code> ）。	门禁卡： <code>public</code> 是通用门禁卡， <code>protected</code> 是家庭成员卡， <code>private</code> 是个人房间钥匙。
函数重写 (Function Overriding)	在派生类中重新定义基类中已存在的同名成员函数。	定制化： 基类提供了一个“通用”功能，派生类可以“重写”它，提供更具体的实现。
虚函数 (Virtual Function)	在基类中使用 <code>virtual</code> 关键字声明的成员函数，用于实现运行时多态。	遥控器： 你有一个“播放”按钮（基类指针），按下它，不同的设备（派生类对象）会执行不同的播放动作。
多态 (Polymorphism)	“多种形态”。通过基类指针或引用调用虚函数时，实际执行的是对象所属派生类的函数版本。	变形金刚： 同一个“汽车人”接口，可以变形为汽车、飞机等不同形态，执行不同的行动。
纯虚函数 (Pure Virtual Function)	在基类中声明但没有实现的虚函数，格式为 <code>virtual void func() = 0;</code> 。它强制派生类必须提供实现。	合同条款： 基类定义了一个“合同”（接口），所有派生类都必须遵守并实现这个功能。
抽象类 (Abstract Class)	包含至少一个纯虚函数的类。不能创建该类的对象。	蓝图： 你不能直接住进一张“蓝图”里，但它为建造具体的房子（派生类）提供了必须遵循的设计。
菱形继承 (Diamond Problem)	在多继承中，一个派生类通过两条或多条路径间接继承了同一个基类，导致基类成员被重复继承，产生二义性。	家谱混乱： 你的曾祖父通过你的祖父和外祖父两条线传给了你，导致你拥有两份曾祖父的“遗产”，造成混乱。

Term	Definition	Memory Anchor / Analogy
虚继承 (Virtual Inheritance)	用于解决菱形继承问题的机制。通过在中间基类继承时使用 <code>virtual</code> 关键字，确保最顶层的基类在最终派生类中只被共享一份实例。	共享账户： 不再每人一份，而是家庭成员共享一个“家庭基金”账户，避免了重复和浪费。

3. Critical Takeaways

中文总结：

- 1. **继承的核心是 “is-a” 关系。** 在决定是否使用继承时，问自己：“派生类”是“基类”的一种吗？例如，“狗”是“动物”的一种。如果不是，应该考虑组合（has-a）。
- 2. **理解访问说明符的权限传递。** `public` 继承最常用，它保持了基类接口的完整性。`protected` 和 `private` 继承会改变基类成员在派生类外部的可见性，主要用于实现细节隐藏。
- 3. **虚函数是实现多态的关键。** 记住，只有通过基类的指针或引用调用虚函数时，才会发生动态绑定（运行时多态）。如果通过对象直接调用，调用的总是该对象所属类的函数版本。
- 4. **纯虚函数和抽象类用于定义接口。** 它们是C++实现“接口”概念的主要方式。一个包含纯虚函数的类不能被实例化，它强制所有派生类提供这些函数的实现，从而保证了接口的一致性。
- 5. **构造函数和析构函数的调用顺序是固定的。** 构造时，先基类后派生类；析构时，先派生类后基类。将基类析构函数声明为 `virtual` 是为了确保在通过基类指针删除派生类对象时，能正确调用派生类的析构函数，防止资源泄漏。

English Summary:

- 1. **The core of inheritance is the "is-a" relationship.** Ask yourself: Is the derived class a type of the base class? E.g., "Dog" is an "Animal". If not, consider composition (has-a) instead.
- 2. **Understand the permission transfer of access specifiers.** `public` inheritance is most common and preserves the base class interface. `protected`

`public` and `private` inheritance change the visibility of base class members outside the derived class, mainly for implementation hiding.

3. **Virtual functions are key to polymorphism.** Remember, dynamic binding (runtime polymorphism) only happens when calling a virtual function through a base class pointer or reference. Calling it directly on an object always calls the function of that object's class.
4. **Pure virtual functions and abstract classes define interfaces.** They are the primary way C++ implements the concept of an interface. A class with a pure virtual function cannot be instantiated, forcing all derived classes to provide an implementation, ensuring interface consistency.
5. **The order of constructor and destructor calls is fixed.** Construction: base class first, then derived class. Destruction: derived class first, then base class. Declaring the base class destructor as `virtual` ensures that when deleting a derived class object via a base class pointer, the derived class destructor is called correctly, preventing resource leaks.

4. Detailed Notes (AI-Expanded)

4.1 继承的基本概念与语法 / Basic Concepts and Syntax of Inheritance

中文:

继承是面向对象编程（OOP）的四大支柱之一（封装、继承、多态、抽象）。它的核心思想是“复用”。当你创建一个新类时，你可以指定它从一个已存在的类“继承”所有的成员变量和成员函数。这个已存在的类被称为**基类**（Base Class）或**父类**（Parent Class），新创建的类被称为**派生类**（Derived Class）或**子类**（Child Class）。

为什么重要？

想象一下，你需要创建 `Dog`、`Cat` 和 `Bird` 三个类。它们都有一些共同的特征，比如 `name`、`age` 和 `eat()`、`sleep()` 函数。如果没有继承，你需要在每个类中重复编写这些代码。这会导致代码冗余、难以维护。通过继承，你可以先创建一个 `Animal` 基类，包含这些通用的属性和方法，然后让 `Dog`、`Cat` 和 `Bird` 继承自 `Animal`。这样，每个派生类只需要编写自己特有的部分（如 `bark()`、`meow()`、`fly()`）。

基本语法:

```
class BaseClass {
public:
    int publicVar;
    void publicMethod() { /* ... */ }
protected:
    int protectedVar;
private:
    int privateVar; // 派生类无法直接访问
};

// 派生类定义
class DerivedClass : public BaseClass {
public:
    void derivedMethod() {
        publicVar = 10; // 可以访问
        protectedVar = 20; // 可以访问
        // privateVar = 30; // 错误! 无法访问
    }
};
```

这里的 `public` 是**访问说明符**，它决定了基类成员在派生类中的访问权限。有三种类型：

- * `public` 继承：基类的 `public` 成员在派生类中仍是 `public`，`protected` 成员仍是 `protected`。这是最常用的方式，它表达了“is-a”关系。
- * `protected` 继承：基类的 `public` 和 `protected` 成员在派生类中都变成 `protected`。
- * `private` 继承：基类的 `public` 和 `protected` 成员在派生类中都变成 `private`。

English:

Inheritance is one of the four pillars of Object-Oriented Programming (OOP) (Encapsulation, Inheritance, Polymorphism, Abstraction). Its core idea is "reuse." When you create a new class, you can specify that it "inherits" all member variables and member functions from an existing class. This existing class is called the **Base Class** or **Parent Class**, and the newly created class is called the **Derived Class** or **Child Class**.

Why is it important?

Imagine you need to create three classes: `Dog`, `Cat`, and `Bird`. They all share common features like `name`, `age`, and functions like `eat()`, `sleep()`. Without inheritance, you would have to write this code repeatedly in each class, leading to redundancy and difficult maintenance. Through inheritance, you can first create a base class `Animal` containing these common attributes and methods, and then have `Dog`, `Cat`, and `Bird` inherit from `Animal`. This way, each derived class only needs to write its own specific parts (e.g., `bark()`, `meow()`, `fly()`).

Basic Syntax:

```
class BaseClass {
public:
    int publicVar;
    void publicMethod() { /* ... */ }
protected:
    int protectedVar;
private:
    int privateVar; // Derived class cannot access directly
};

// Derived class definition
class DerivedClass : public BaseClass {
public:
    void derivedMethod() {
        publicVar = 10; // Accessible
        protectedVar = 20; // Accessible
        // privateVar = 30; // Error! Not accessible
    }
};
```

The `public` here is the **access specifier**, which determines how the base class members are accessible in the derived class. There are three types:

- * `public` inheritance: Base class `public` members remain `public` in the derived class, and `protected` members remain `protected`. This is the most common way, expressing an "is-a" relationship.
- * `protected` inheritance: Both `public` and `protected` members of the base class become `protected` in the derived class.

* `private` inheritance: Both `public` and `protected` members of the base class become `private` in the derived class.

4.2 继承类型 / Types of Inheritance

中文:

C++ 支持多种继承方式，以适应不同的设计需求。

1. **单继承 (Single Inheritance):** 一个派生类只有一个直接基类。这是最简单、最常用的形式。

A | B

2. **多继承 (Multiple Inheritance):** 一个派生类可以有多个直接基类。这提供了强大的能力，但也可能引入复杂性，如“菱形继承”问题。

A B \ / C

3. **多层继承 (Multilevel Inheritance):** 一个类从另一个派生类派生，形成一条链。

A | B | C

4. **层次继承 (Hierarchical Inheritance):** 多个派生类继承自同一个基类。

A / | \ B C D

5. **混合继承 (Hybrid Inheritance):** 上述类型的组合。例如，结合了多层继承和层次继承。这通常会导致菱形继承问题。

菱形继承问题 (The Diamond Problem):

当使用多继承时，如果两个基类都继承自同一个更顶层的基类，那么最终派生类会包含两份顶层基类的成员，导致二义性。

```
GrandParent
 /      \
Parent1  Parent2
 \      /
  Child
```

解决方案：虚继承 (Virtual Inheritance)

通过在中间类 (Parent1 和 Parent2) 继承 GrandParent 时使用 `virtual` 关键字，可以确保 GrandParent 的成员在 Child 类中只被共享一份，从而解决二义性。

```
class Parent1 : virtual public GrandParent { ... };  
class Parent2 : virtual public GrandParent { ... };  
class Child : public Parent1, public Parent2 { ... };
```

English:

C++ supports several types of inheritance to suit different design needs.

1. **Single Inheritance:** A derived class has only one direct base class. This is the simplest and most common form.
2. **Multiple Inheritance:** A derived class can have multiple direct base classes. This provides great power but can introduce complexity, such as the "diamond problem".
3. **Multilevel Inheritance:** A class is derived from another derived class, forming a chain.
4. **Hierarchical Inheritance:** Multiple derived classes inherit from the same base class.
5. **Hybrid Inheritance:** A combination of the